BHAVAN'S COLLEGE AUTONOMOUS, ANDHERI-WEST
PRACTICAL JOURNAL

Class: TYIT **Sem: V** **Roll No.: TYIT-11** **Date : / /2026**

Course Name: Big Data Analytics **Page no:**

Practical Number:- 2

Aim :- Implement word count program using MapReduce.

Commands

- **start-dfs.sh** : A Bash script used in Apache Hadoop to start the HDFS daemons, specifically the namenode, datanodes, and secondary namenode.
- **start-yarn.sh:** A utility command in the Hadoop ecosystem used to start the YARN (Yet Another Resource Negotiator) daemons, specifically the ResourceManager and NodeManagers, on all designated hosts configured in the cluster.
- **jps:** lists running Java processes on a local or remote system.
- **mkdir directory_name :** Creates a directory with the given directory_name (a folder is called 'directory' in linux lingo, saying something as a 'folder' in linux is a grave insult of the linux gods and gets you cursed with errors /s).
- **cd directory_name:** Opens the created directory in the terminal.
- **nano Mapper.py:** Nano is a terminal text editor (just like VS Code or notepad but it runs inside the terminal) built into most linux distros, it checks for the given file in current directory and automatically creates an empty file if it doesn't already exist and opens it. Here we created the Mapper script for mapreduce job
- **Enter following Code for Mapper.py:**

```
#!/usr/bin/env python3 import sys
for line in sys.stdin:
  line=line.strip().lower()
  words = line.split(' ')
for word in words:
  if word: print(f"{word}\t1")
```

Note: the #!/usr/bin/env python3 in the first line is a shebang (or hashbang) , it is the character sequence #! placed at the very first line of a Python script to specify the interpreter path required to execute the file. This feature is primarily used on Unix-like systems (Linux, macOS) to allow scripts to run directly from the command line without explicitly typing python or python3.

- Hit ctrl+s to save the changes, and ctrl+x to close nano and get back on terminal.
- **nano Reducer.py:** Create and open Reducer.py which will act as the Reducer script for mapreduce job.
- **Enter following Code for Reducer.py:**

```
#!/usr/bin/env python3 import sys
current_word = None
current_count =0
for line in sys.stdin:
 try:
  word, count = line.split('\t', 1)
  count = int(count)
except Exception as
  e: print(f"{e}\n")
continue
  if current_word ==word:
  current_count += count
else:
  if current_word:
```

Teacher's Signature

BHAVAN'S COLLEGE AUTONOMOUS, ANDHERI-WEST
PRACTICAL JOURNAL

Class: TYIT **Sem: V** **Roll No.: TYIT-11** **Date : / /2026**

Course Name: Big Data Analytics **Page no:**

Practical Number:- 2

```
  print(f"{current_word}\t{current_count}")
  current_count=count current_word =word
if current_word is not None:
  print(f"{current_word}\t{current_count}")
```

- **echo -e "hello Ravit \nhello TYIT-11" > input.txt:** echo command is basically like a print statement in terminal which prints the content inside " ", the -e flag tells terminal to enable interpreting of escape sequences like \n, without it \n will be printed instead of new line, > tells terminal to write the output of previous command to the given file , and create the said file if it doesn't already exist.
- **cat input.txt | python3 mapper.py | sort | python3 reducer.py :** The cat command, short for concatenate, is a fundamental utility in Linux used to display file contents, create new files, and combine multiple files. sort is the command to sort the given input . The '|' symbol aka the pipe is used to tell that the output of command at left side should be used as input for command at right. So the flow here is ,
  1. Output of cat input.xt > Input for mapper (accessed with help of sys.stdin) .
  2. Output of Mapper.py > Input for sort.
  3. Output of sort > Input for reducer.py
  4. Output of Reducer.py is displayed as final output.

  This is an optional step, used to simulate the working of a mapreduce task and check if there's any error with any of mapper or reducer files.
- **hdfs dfs -mkdir -p /user/path/for/directory:** The hdfs dfs command is the primary utility for executing file system operations on the Hadoop Distributed File System (HDFS), -mkdir flag tells it's a make directory command, -p flag is a flag under mkdir that tells that create any parent folders that are present in path but don't exist , eg : if /user/path exists but for directory doesn't , a directory name for is created and another directory will be created inside our 'for' directory.
- **Hdfs dfs -put mapper.py reducer.py input.txt /user/path/for/directory:** Copies mapper.py, reducer.py and input.py and adds them into the folder we created in previous step.
- **hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -input /user/ubuntu/TYIT-11_WordCount/input.txt -output /user/ubuntu/TYIT-11_WordCount/output - mapper mapper.py -reducer reducer.py -file mapper.py -file reducer.py :** hadoop jar command is used to execute java based hadoop application , the path followed by hadoop jar is the path of application we want to execute , in this case hadoop-streaming-3.3.6.jar , which is , the Hadoop Streaming bridge , a Java wrapper that lets us write MapReduce logic in any language that reads from stdin and writes to stdout. -input flag tells thein hdfs path to file to used as input, in this case the input.txt we put in previous step. - output flag tells the path to directory which should be created to store output. -mapper flag tells the mapper file , -reducer tells the reducer file to be used. -file flags tell the files to be shipped to all cluster node (without these flags , the mapper and reducer won't reach the worker nodes leading to file not found error and streaming job failed).

  The output will be stored in /user/ubuntu/TYIT-11_WordCount/output/part-r-00000 (one file per reducer task; by default there's 1 reducer).

  Note:
  1. The output file must not exist in hadoop already, if it does then create a directory with different name or delete the existing one using hdfs dfs -rm -r /user/path/to/output command.
  2. The /user/ubuntu/TYIT-11_WordCount is the directory we created in step 13.
  3. hdfs dfs -ls /user/path/to/output : -ls is the list command , it lists the contents of given directory. The path to output should be same as given in previous step . This step is optional and is done to check the contents of output file and look for _SUCCESS and part-00000 file.

Teacher's Signature

BHAVAN'S COLLEGE AUTONOMOUS, ANDHERI-WEST
PRACTICAL JOURNAL

Class: TYIT **Sem: V** **Roll No.: TYIT-11** **Date : / /2026**

Course Name: Big Data Analytics **Page no:**

Practical Number:- 2

4. hdfs -dfs -cat /user/ubuntu/TYIT-11_Wordcount/output/part-00000 : -cat flag refers to cat command of linux similar to the one in Step 12. this command is used to check the output of our streaming job from step 15

Output:

```
ubuntu@ubuntu2004:~/TYIT-11_WordCount$ hdfs dfs -cat /user/ubuntu/TYIT-11_Word/output/part-0000
hello 2
Ravit 1
TYIT-11 1
ubuntu@ubuntu2004:~/TYIT-11_WordCount$
```

Teacher's Signature